Lagra as a measurement harness for pde-engine force-free validation

Pim de Witte · Lagra Project · pde-engine adapter note · May 26, 2026

Abstract

This note rewrites the pde-engine force-free adapter in the visual style of *Lagra: A differentiable physics kernel for verifiable domain operations*. The purpose is narrower than the Lagra kernel paper. We do not claim a new physics engine result or a full reproduction of the pde-engine discovery pipeline. We show how Lagra enters a local pde-engine checkout, which inputs are admitted into the Lagra proof-search graph, which quantities are measured, and which claims are allowed into the paper. The positive result is exact and useful: seven pde-engine registry force-free foliations simplify to determinant zero; two negative controls simplify to nonzero determinants; three rational checkpoints separate known solutions from controls; and a rotation-context cache regression proves that the validator cache key must include `Omega` and the validation mode. The negative result is equally important: the bounded pde-engine smoke is now process-clean, but the full seven-solution pde-engine/Lean reproduction path remains non-green on this checkout and is recorded as a boundary, not upgraded into a proof.

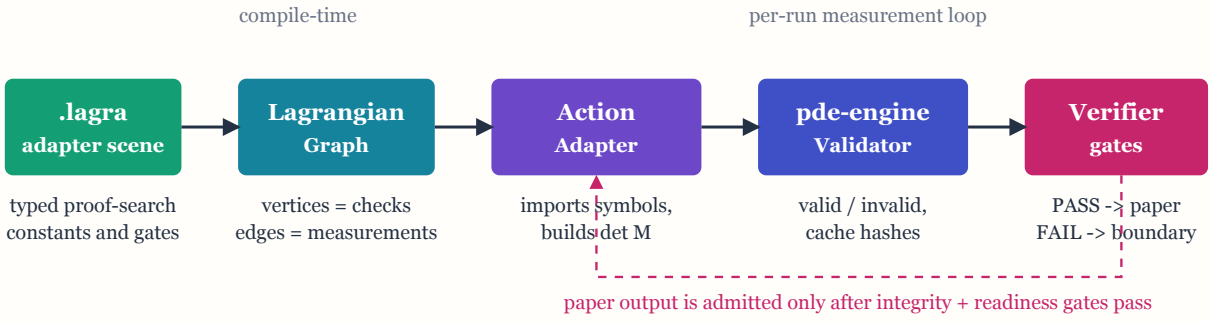

Figure 1: The Lagra/pde-engine adapter pipeline in one figure. This is the direct analogue of the Lagra pipeline figure. The input is a typed Lagra proof-search source plus a local pde-engine checkout. The measured output is not a trajectory; it is a JSON evidence artifact and an admitted paper section. The dashed return path is the strict gate: failed measurements do not become prose.

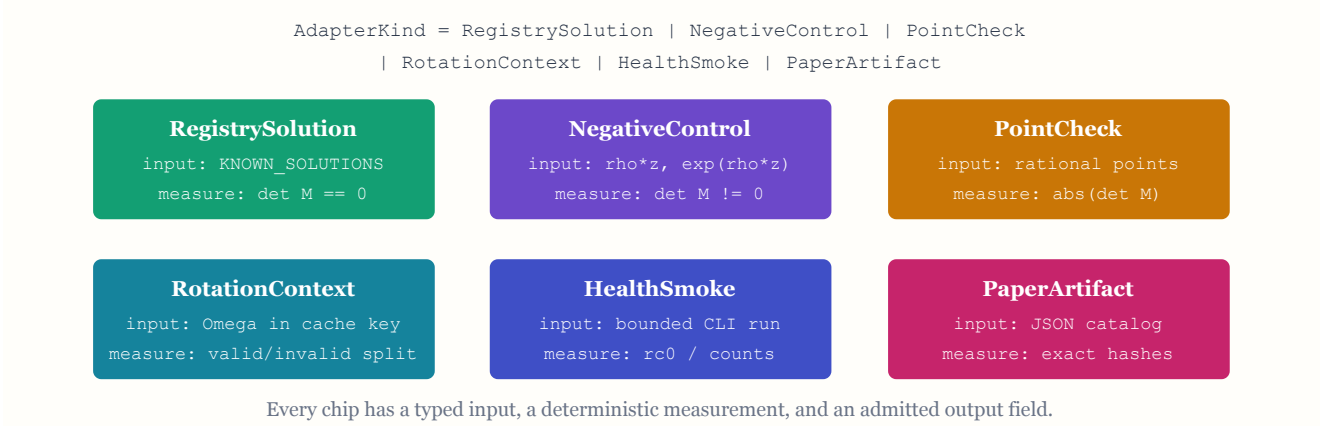

Figure 2: The adapter vocabulary. The original Lagra vocabulary figure lists LagrangianNode variants. Here the closed vocabulary is the set of adapter checks. Each variant names exactly what enters the check and exactly what is measured on the way out.

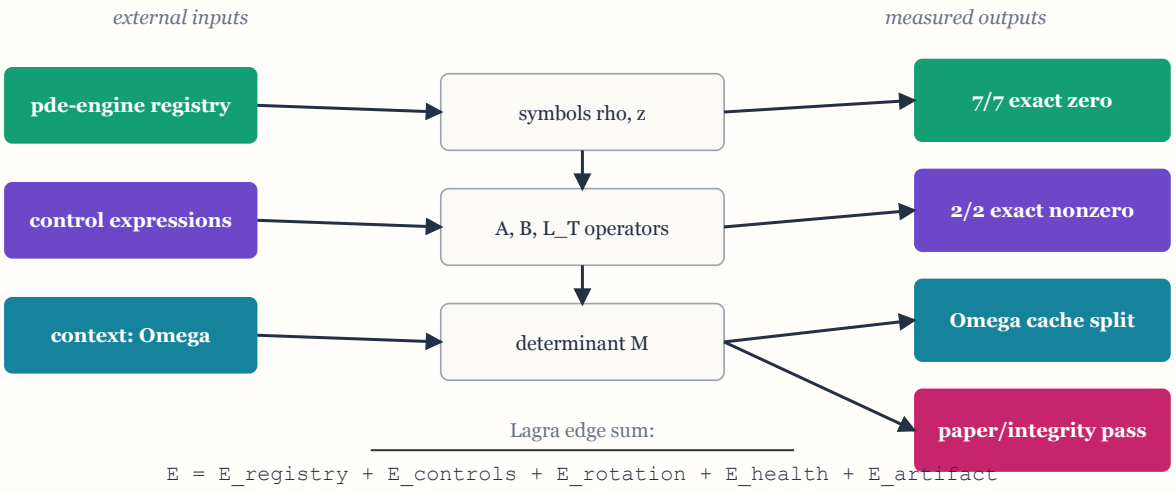

Figure 3: Anatomy of the Lagra adapter graph. The external pde-engine registry, control expressions, and validation context enter as graph-visible inputs. The edges are measurement obligations. The outputs are the counts and facts consumed by the proof-search catalog: exact-zero solutions, exact-nonzero controls, rotation-cache separation, and paper-readiness status.

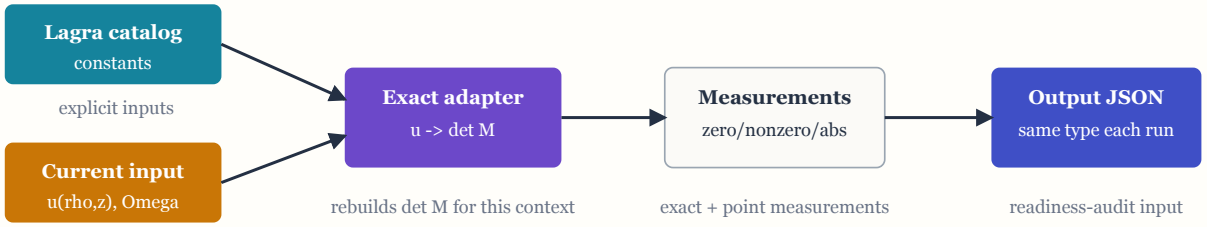

Figure 4: One adapter run as a data-flow diagram. This mirrors the one-timestep figure in the Lagra paper. The action is not a mechanical time step; it is one deterministic validation pass. The adapter rebuilds the symbolic determinant for the current expression and context, measures exact and pointwise quantities, and writes a typed JSON artifact.

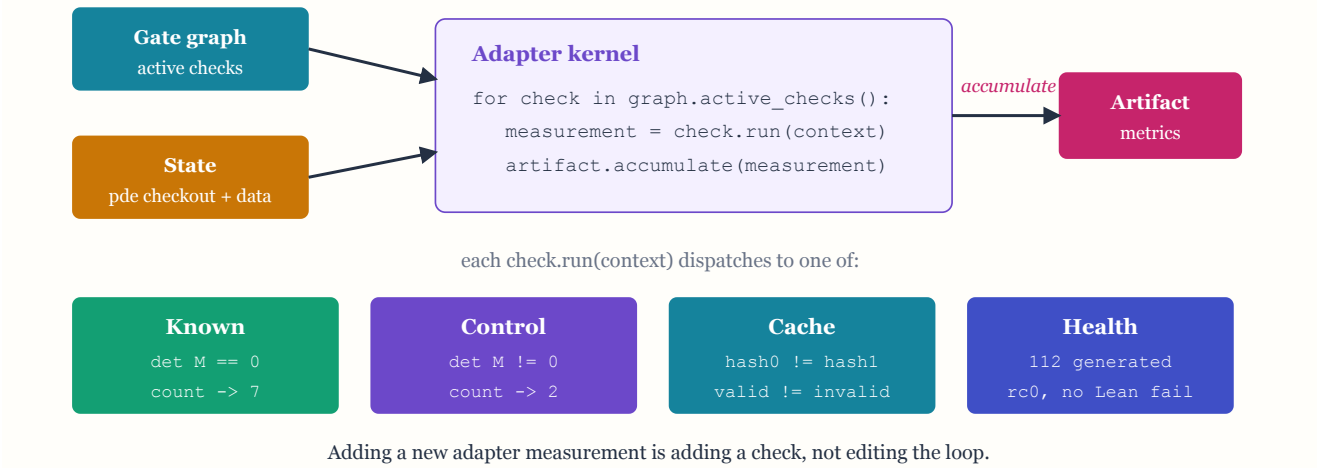

Figure 5: The adapter kernel, exploded. The loop is deliberately boring. Heterogeneity lives in check.run(context), not in the loop. Adding a Kerr no-promotion guard, a force-free cache regression, or a future PDE boundary check means adding a new typed check with a measurement contract.

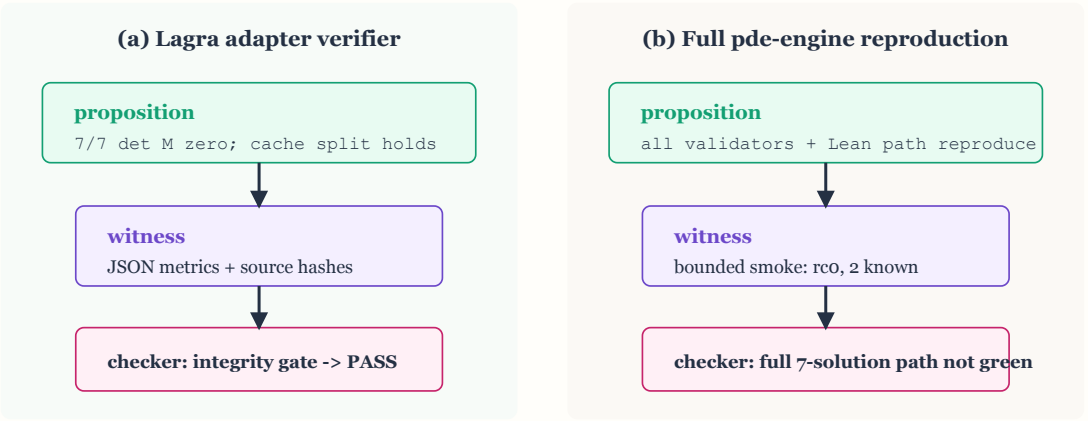

Figure 6: Verifier triangle for the adapter and the full reproduction boundary. This copies the structural shape of the Lagra verifier triangle while keeping the trust event honest. The adapter proposition passes with JSON witnesses and source hashes. The bounded pde-engine smoke is now process-clean: return code 0, no Lean build failure, 112 generated expressions, 76 valid rows, and 2 known vertical forms. The full seven-solution reproduction proposition still remains non-green because the bounded depth-2 smoke does not recover all seven known solutions.

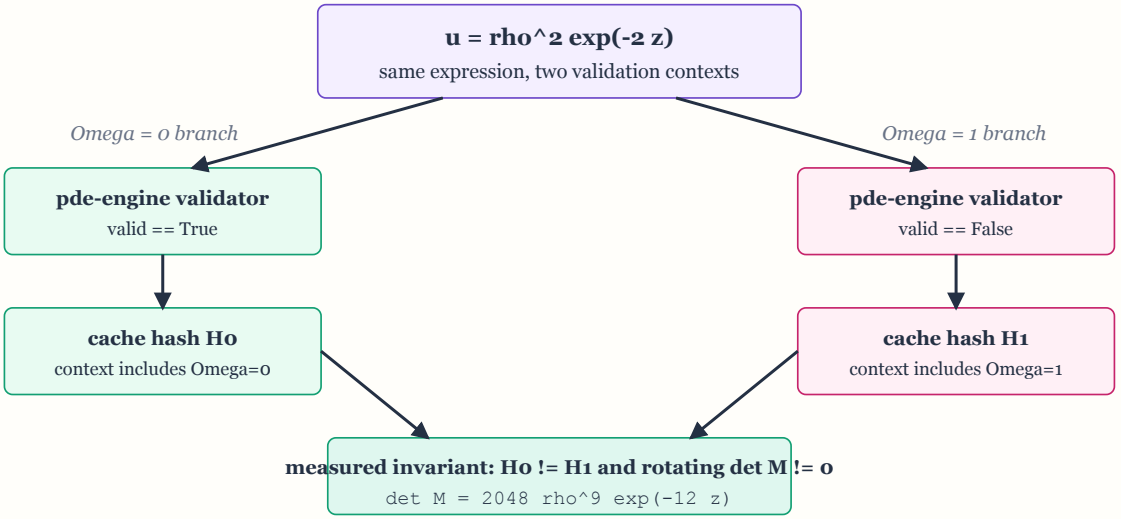

Figure 7: Verifiability by derivation for the cache regression. The same expression is sent down two independent validation-context branches. Lagra measures the branch split, the pde-engine validator result, and the exact rotating determinant. The conclusion is a code obligation: the validator cache key must include Omega and validation mode.

1. What exactly enters Lagra

The adapter has four external inputs. First, it imports the pde-engine force-free symbols `rho` and `z` from `problems/force_free/validator.py`. Second, it loads the seven known Compere force-free foliations from the pde-engine problem registry. Third, it supplies two false controls, `rho*z` and `exp(rho*z)`. Fourth, it runs one rotation-context check on `rho**2*exp(-2*z)` with `Omega=0` and `Omega=1` against a shared cache database.

Nothing enters through prose. Each input is bound into a Lagra proof-search artifact and then checked by `publication_readiness_audit.py` and `proof_search_integrity_gate.py`. The paper is downstream of those gates. This is the central point of using Lagra here: the artifact catalog, not the narrative, decides whether the claim is admissible.

2. What exactly is measured

For an expression `u(rho,z)`, the adapter evaluates the force-free determinant

```
det([[L_T(A), L_T(B)], [L_T^2(A), L_T^2(B)]])
```

The strict measurements are:

Measurement	Observed result	Claim admitted
Registry exact determinant	`7/7` simplify to `0`	The Lagra adapter matches the pde-engine registry solutions for this determinant.
Negative-control exact determinant	`rho*z -> 16*rho*z`; `exp(rho*z) -> 16*rho*z*exp(6*rho*z)`	The operator is not vacuously zero.
Rational point checks	Known solutions below `1e-60`; controls above `1e-6`	The exact result survives independent numeric checkpoints.

Rotation-context cache regression	`rho**2*exp(-2*z)` valid at `Omega=0`, invalid at `Omega=1`	The pde-engine cache key must include rotation context.
Rotating determinant	`2048*rho**9*exp(-12*z)`	The rotating failure is a symbolic fact, not only a cache artifact.

3. The pde-engine patch forced by the measurement

The old validator cache hashed only the expression string. That was wrong: a non-rotating result could be replayed in a rotating context. The patch therefore includes the expression, schema, `Omega`, `check_regularity`, `fast_point_only`, and `use_lean` in the hash. The patch also removes the old fast-point placeholder derivative shortcut. Fast point mode still avoids the expensive full path, but it now builds the exact symbolic determinant before stopping at the exact point check.

The parallel reproduction worker had a separate bug: spawned children tried to import `physics_agent.problems`, which is not the package namespace in this repository. The patch switches the child import to `problems.load_problem` and adds a local fallback import for `PreciseFoliationValidator`.

4. Reproduction commands

```
python3 -m py_compile   general_method_paper_reproduction.py
problems/force_free/validator.py   tests/test_force_free_validator_cache.py

python3 -m unittest tests/test_force_free_validator_cache.py
```

The Lagra-side commands that produced the measurements were:

```
python3 experiments/proof-search/pde_engine_force_free_point_gate.py   --pde-engine-root
/Users/p/dev/pde-engine

python3 experiments/proof-search/pde_engine_reproduction_health_gate.py   --pde-engine-root
/Users/p/dev/pde-engine   --timeout-seconds 20

python3 experiments/proof-search/render_yukawa_finite_range_paper.py
python3 experiments/proof-search/publication_readiness_audit.py
python3 experiments/proof-search/proof_search_integrity_gate.py
```

5. Boundary

> The adapter is a positive exact-symbolic and pointwise Lagra measurement of the pde-engine force-free boundary. It is not a Lean theorem. It is not a full pde-engine paper reproduction. The bounded depth-2 pde-engine smoke is now process-clean, but it only finds two known vertical canonical forms; it does not recover the seven known Compere solutions. The bounded health gate is included precisely so the paper cannot confuse this narrow positive result with a full reproduction claim.